

30-Day Agentic AI Roadmap

Tailored for Pramod Jadhav's automation engineering background

Why this PDF is worth using

This roadmap is long and practical. A clickable agenda makes it much easier to use as a daily execution guide instead of a one-time reading document. For your profile, that matters because the plan is tied to two real portfolio projects and interview preparation, not generic AI theory.

Outcome by Day 30

Main project	Automation Support Agent
Mini project	Log Analysis Agent
DevOps layer	Docker, GitHub Actions, deployment
Interview deliverables	Notes, architecture diagrams, project walk-throughs

Recommended Daily Execution Sequence

Use the same sequence every day so the plan stays practical and repeatable. For your background, the right balance is less passive study and more hands-on building.

Step	Time	What to do
1. Quick review	10 to 15 min	Open yesterday's notes, check what was completed, and decide today's single main target.
2. Learning	45 to 60 min	Read only the material for that day. Capture short notes while learning, but avoid over-studying.
3. Practical build	60 to 90 min	Implement exactly what you studied. Build small, test fast, and keep momentum.
4. Debug and improve	20 to 40 min	Fix errors, rerun the code, simplify if needed, and get to a working state.
5. Notes and interview prep	20 to 30 min	Write what you learned, what broke, how you fixed it, and how to explain it in an interview.
6. End-of-day closure	5 to 10 min	Commit code, update the checklist, and note tomorrow's starting point.

Daily rule: Follow this order every day: **Review -> Learn -> Build -> Debug -> Notes -> Commit**. Do not spend more time studying than building. If you get stuck, try on your own for 15 to 20 minutes, then check docs, simplify the task, and move forward.

Realistic daily time: around 3 hours on average. Normal study days may take 2.5 to 3 hours, while build, Docker, CI, and deployment days may take 3.5 to 4 hours.

Agenda

[Day 1: Understand The Landscape](#)

[Day 2: Prompt Basics](#)

[Day 3: First API Calls](#)

Day 4: Learn The Agent Loop

Day 5: Interview Framing

Day 6: Build A Simple Chatbot

Day 7: Week 1 Review

Day 8: Choose Framework And Project Shape

Day 9: Define Enterprise Requirements

Day 10: Build The Issue Classifier

Day 11: Build Basic Diagnostic Tools

Day 12: Connect Classifier To Tools

Day 13: Build Structured Support Summaries

Day 14: Prepare Realistic Sample Cases

Day 15: Add Approval Gate

Day 16: Strengthen Prompts For Enterprise Use

Day 17: Add Evaluation Cases

Day 18: Start Project 2 - Log Analysis Agent

Day 19: Build A Log Parser

Day 20: Add LLM-Based RCA Summary

Day 21: Polish Both READMEs

Day 22: Learn Docker Basics

Day 23: Run Project 1 In Docker

Day 24: Dockerize Project 2

Day 25: Add GitHub Actions CI

Day 26: Learn Hugging Face Spaces Deployment

Day 27: Deploy One Project

Day 28: Context Engineering Pass

Day 29: Interview And System Design Preparation

Day 30: Final Polish And Portfolio Pack

Day 1: Understand The Landscape

Goal: Set up the environment and understand LLM, workflow, agent, and RAG fundamentals.

Study material

- [Python for Beginners](#)
- [Hugging Face AI Agents Course Unit 0](#)

Exact steps

- Install Python 3.11+, VS Code, Git, and Docker Desktop.
- Create the workspace folder `enterprise-agentic-ai` and a virtual environment.
- Create `notes/` and `projects/` folders and a learning journal file.
- Write notes in your own words for: LLM, workflow, agent, RAG, and where your automation background fits.

Suggested commands

```
mkdir enterprise-agentic-ai
cd enterprise-agentic-ai
python3 -m venv .venv
source .venv/bin/activate
mkdir notes projects
touch notes/interview-notes.md notes/learning-journal.md notes/day1-foundations.md
```

End-of-day output: A ready local environment and `notes/day1-foundations.md`.

Interview focus: Prepare a 2-minute explanation of why your automation background is a good fit for agentic AI.

[Back to agenda](#)

Day 2: Prompt Basics

Goal: Learn how to write clear prompts using real enterprise support scenarios.

Study material

- [OpenAI Prompt Best Practices](#)

Exact steps

- Create 10 prompt examples from your real use cases: low disk space, upgrade failure, Citrix issue, package install failure, notarization failure.
- For each case, write one weak prompt and one improved prompt.
- Capture patterns that make the improved prompt better: context, constraints, format, and safety.

Suggested commands

```
mkdir -p notes
touch notes/day2-prompts.md
code notes/day2-prompts.md
```

End-of-day output: `notes/day2-prompts.md` with 10 prompt pairs.

Interview focus: Be ready to explain what makes a prompt production-safe versus demo-only.

[Back to agenda](#)

Day 3: First API Calls

Goal: Use the OpenAI API directly and build confidence with Python scripts.

Study material

- [OpenAI Python Quickstart](#)

Exact steps

- Install the `openai` Python package in your virtual environment.
- Export `OPENAI\_API\_KEY` securely in your shell session.
- Create `simple\_completion.py`, `classify\_issue.py`, and `summarize\_log.py` under `projects/playground/`.
- Run each script and save a sample input/output pair in your notes.

Suggested commands

```
source .venv/bin/activate
pip install openai
mkdir -p projects/playground
touch projects/playground/simple_completion.py projects/playground/classify_issue.py
projects/playground/summarize_log.py
export OPENAI_API_KEY='your_key_here'
python projects/playground/simple_completion.py
```

End-of-day output: Three working scripts that call the model successfully.

Interview focus: Prepare to explain how you would secure API keys in a real project.

[Back to agenda](#)

Day 4: Learn The Agent Loop

Goal: Understand observe-think-act-tool-answer flow and map it to enterprise automation.

Study material

- [Hugging Face Agents Course](#)
- [OpenAI Agents Guide](#)

Exact steps

- Read the core explanations of what an agent does differently from a static script.
- Draw a simple flow: user input -> classifier -> tool call -> summarizer -> recommendation.
- Map each step to a real use case from your background, such as upgrade readiness or package troubleshooting.

Suggested commands

```
touch notes/day4-agent-loop.md
code notes/day4-agent-loop.md
```

End-of-day output: `notes/day4-agent-loop.md` plus one simple architecture diagram.

Interview focus: Prepare to explain why tool use matters in enterprise support automation.

[Back to agenda](#)

Day 5: Interview Framing

Goal: Turn the basic concepts into interview-ready explanations.

Study material

- [OpenAI Agents Guide](#)

Exact steps

- Write short answers for: What is an agent? Workflow vs agent? When should you not use an agent?
- Add a section explaining why enterprise automation is a strong use case for agentic AI.
- Keep all answers simple enough to speak aloud in 60 to 90 seconds.

Suggested commands

```
code notes/interview-notes.md
```

End-of-day output: Section 1 added to `notes/interview-notes.md`.

Interview focus: Practice speaking the answers without reading them word-for-word.

[Back to agenda](#)

Day 6: Build A Simple Chatbot

Goal: Get comfortable with a basic conversational loop before building agents.

Study material

- [OpenAI Python Quickstart](#)

Exact steps

- Create `projects/chatbot/chatbot.py`.
- Add a loop that accepts user input and sends it to the model.
- Set one system prompt that defines the chatbot's role clearly.
- Test at least 5 prompts from your Day 2 prompt practice file.

Suggested commands

```
mkdir -p projects/chatbot
touch projects/chatbot/chatbot.py
python projects/chatbot/chatbot.py
```

End-of-day output: A working CLI chatbot.

Interview focus: Be ready to explain why a chatbot is not the same thing as an agent.

[Back to agenda](#)

Day 7: Week 1 Review

Goal: Consolidate learning and make sure all basics are clear.

Study material

- [Python for Beginners](#)
- [Hugging Face AI Agents Course](#)

Exact steps

- Push all Week 1 code and notes to GitHub.
- Review your notes and tighten any weak definitions.
- Record yourself answering five basic interview questions.

Suggested commands

```
git init
git add .
git commit -m 'Week 1 foundations complete'
git branch -M main
```

```
git remote add origin <your_repo_url>
git push -u origin main
```

End-of-day output: `notes/week1-review.md` and first GitHub push.

Interview focus: You should now be able to define LLM, RAG, workflow, and agent without confusion.

[Back to agenda](#)

Day 8: Choose Framework And Project Shape

Goal: Set up the main project using a framework appropriate for your background.

Study material

- [smolagents Docs](#)
- [OpenAI Agents SDK Quickstart](#)

Exact steps

- Read both framework overviews and compare simplicity versus control.
- Choose OpenAI Agents SDK for the main project and use smolagents only as optional reference.
- Create the main project folder `projects/automation\_support\_agent` with initial files: `app.py`, `tools.py`, `prompts.py`, `report.py`, `README.md`, `requirements.md`.

Suggested commands

```
pip install openai-agents smolagents
mkdir -p projects/automation_support_agent
touch projects/automation_support_agent/README.md projects/automation_support_agent/app.py
projects/automation_support_agent/tools.py projects/automation_support_agent/prompts.py
projects/automation_support_agent/report.py projects/automation_support_agent/requirements.md
```

End-of-day output: Main project skeleton created.

Interview focus: Write one note on why you chose this framework for your use case.

[Back to agenda](#)

Day 9: Define Enterprise Requirements

Goal: Specify the MVP like a real internal product.

Study material

- [OpenAI Agents Guide](#)

Exact steps

- In `requirements.md`, define issue categories: disk, upgrade, packaging, application, unknown.
- List safe read-only tools and risky remediation actions.
- Define required outputs: issue category, findings, recommendation, approval needed, support summary.

Suggested commands

```
code projects/automation_support_agent/requirements.md
```

End-of-day output: A complete MVP requirements document.

Interview focus: Practice explaining the difference between an MVP and a production-ready internal agent.

[Back to agenda](#)

Day 10: Build The Issue Classifier

Goal: Classify support issues before any tool is invoked.

Study material

- [OpenAI Prompt Best Practices](#)

Exact steps

- Implement issue classification in `app.py`.
- Support at least five categories.
- Add 10 sample problem statements and verify that the classifier behaves reasonably.

Suggested commands

```
code projects/automation_support_agent/app.py
python projects/automation_support_agent/app.py
```

End-of-day output: A working classifier function plus sample test cases.

Interview focus: Be ready to discuss false classifications and how you would handle them.

[Back to agenda](#)

Day 11: Build Basic Diagnostic Tools

Goal: Create safe tools that the future agent can call.

Study material

- [Python for Beginners](#)

Exact steps

- Implement mocked safe tools in `tools.py`: `check\_disk\_space()`, `check\_os\_version()`, `check\_package\_status()`, `read\_sample\_log(path)`.
- Keep them simple and deterministic first.
- Print structured results so the agent can consume them easily.

Suggested commands

```
code projects/automation_support_agent/tools.py
python -c "from projects.automation_support_agent.tools import check_disk_space"
```

End-of-day output: Basic tool layer working locally.

Interview focus: Prepare to explain why agents should use tools instead of free-form guessing.

[Back to agenda](#)

Day 12: Connect Classifier To Tools

Goal: Create the first end-to-end flow of your main project.

Study material

- [OpenAI Agents SDK Quickstart](#)

Exact steps

- Route each issue category to a specific diagnostic tool.
- Return issue category, evidence, and next recommended action.
- Test at least 5 different support scenarios.

Suggested commands

```
python projects/automation_support_agent/app.py
```

End-of-day output: Basic end-to-end automation support flow.

Interview focus: Prepare to walk through the flow step by step.

[Back to agenda](#)

Day 13: Build Structured Support Summaries

Goal: Generate support-ready outputs instead of raw AI text.

Study material

- [OpenAI Platform Docs](#)

Exact steps

- Use `report.py` to create a structured summary payload.
- Include issue, category, tools used, findings, recommendation, and approval-needed flag.
- Save sample JSON outputs for 3 scenarios.

Suggested commands

```
code projects/automation_support_agent/report.py
python projects/automation_support_agent/app.py
```

End-of-day output: Structured support report generation.

Interview focus: Be ready to explain why structured outputs are easier to integrate than plain paragraphs.

[Back to agenda](#)

Day 14: Prepare Realistic Sample Cases

Goal: Make the project look grounded in real enterprise operations.

Study material

- [OpenAI Agents Guide](#)

Exact steps

- Create sample input files: `disk\_issue.txt`, `upgrade\_failure.log`, `package\_install.log`, `citrix\_issue.log`.
- Use patterns from your past work, but keep sensitive details out.
- Run the project against each sample file and note weak spots.

Suggested commands

```
mkdir -p projects/automation_support_agent/sample_data
touch projects/automation_support_agent/sample_data/disk_issue.txt
touch projects/automation_support_agent/sample_data/upgrade_failure.log
touch projects/automation_support_agent/sample_data/package_install.log
touch projects/automation_support_agent/sample_data/citrix_issue.log
```

End-of-day output: Realistic sample data and first project test run notes.

Interview focus: This gives you stronger material when interviewers ask if the project reflects real work.

[Back to agenda](#)

Day 15: Add Approval Gate

Goal: Introduce the safety model that matters in enterprise automation.

Study material

- [OpenAI Agents Guide](#)

Exact steps

- Mark read-only tools as auto-runnable.
- Mark cleanup, reinstall, and deletion-like actions as approval-required.
- Prompt the user for confirmation before risky execution.

Suggested commands

```
code projects/automation_support_agent/app.py
python projects/automation_support_agent/app.py
```

End-of-day output: Safety gate added to the main project.

Interview focus: Prepare a good answer for 'Why should AI not directly run all scripts?'

[Back to agenda](#)

Day 16: Strengthen Prompts For Enterprise Use

Goal: Reduce hallucination risk and keep the model grounded in evidence.

Study material

- [OpenAI Prompt Best Practices](#)

Exact steps

- Update `prompts.py` so the model must only use available evidence.
- Add clear fallback behavior: say when information is missing.
- Prefer safest next action first and avoid over-claiming.

Suggested commands

```
code projects/automation_support_agent/prompts.py
```

End-of-day output: Improved enterprise-safe prompt set.

Interview focus: Be ready to explain how you reduce hallucinations in automation scenarios.

[Back to agenda](#)

Day 17: Add Evaluation Cases

Goal: Evaluate the project like an engineer, not just a demo builder.

Study material

- [Introducing AgentKit](#)

Exact steps

- Create `eval\_cases.md` with 15 scenarios.
- Define expected category, tool, and acceptable recommendation for each.
- Run the project manually and record pass/fail.

Suggested commands

```
touch projects/automation_support_agent/eval_cases.md
code projects/automation_support_agent/eval_cases.md
```

End-of-day output: Manual evaluation sheet for Project 1.

Interview focus: Prepare to answer how you measure agent quality.

[Back to agenda](#)

Day 18: Start Project 2 - Log Analysis Agent

Goal: Build a second project that is tightly aligned to your automation background.

Study material

- [Hugging Face Context Course](#)

Exact steps

- Create `projects/log\_analysis\_agent` with `app.py`, `parser.py`, `prompts.py`, and `README.md`.
- Define the problem: analyze automation logs and produce RCA plus next steps.

Suggested commands

```
mkdir -p projects/log_analysis_agent
touch projects/log_analysis_agent/app.py projects/log_analysis_agent/parser.py
projects/log_analysis_agent/prompts.py projects/log_analysis_agent/README.md
```

End-of-day output: Second project skeleton.

Interview focus: This project will help you talk about AI for diagnostics, not just support chat.

[Back to agenda](#)

Day 19: Build A Log Parser

Goal: Detect common failure patterns before adding LLM summarization.

Study material

- [Python for Beginners](#)

Exact steps

- Support patterns like permission denied, disk full, timeout, signing issue, package failure.
- Return matched labels plus the lines where they were found.
- Test the parser on at least 5 sample logs.

Suggested commands

```
code projects/log_analysis_agent/parser.py
python projects/log_analysis_agent/parser.py
```

End-of-day output: Working parser with pattern detection.

Interview focus: Explain why combining deterministic parsing with LLM reasoning is stronger than using the LLM alone.

[Back to agenda](#)

Day 20: Add LLM-Based RCA Summary

Goal: Use parser evidence plus LLM reasoning to generate concise root cause analysis.

Study material

- [OpenAI Python Quickstart](#)

Exact steps

- Pass parser results and a short raw log excerpt to the model.

- Return probable root cause, confidence, and next action.
- Save at least 3 sample outputs in your notes.

Suggested commands

```
code projects/log_analysis_agent/app.py
python projects/log_analysis_agent/app.py
```

End-of-day output: A working Log Analysis Agent.

Interview focus: This is a good example to discuss hybrid deterministic + LLM design.

[Back to agenda](#)

Day 21: Polish Both READMEs

Goal: Make both projects portfolio-ready.

Study material

- [Getting started with Git](#)

Exact steps

- For both projects, document the problem, architecture, tools, limitations, and next steps.
- Add a short setup guide and example usage commands.

Suggested commands

```
code projects/automation_support_agent/README.md
code projects/log_analysis_agent/README.md
git add . && git commit -m 'Project READMEs polished'
```

End-of-day output: Interview-friendly READMEs for both projects.

Interview focus: A good README makes you look organized and production-minded.

[Back to agenda](#)

Day 22: Learn Docker Basics

Goal: Start the minimum DevOps layer needed for project credibility.

Study material

- [Docker Get Started](#)
- [Docker Introduction](#)

Exact steps

- Understand image, container, Dockerfile, build, and run.
- Write a first Dockerfile for the main project.
- Keep dependencies and startup command simple.

Suggested commands

```
touch projects/automation_support_agent/Dockerfile
code projects/automation_support_agent/Dockerfile
```

End-of-day output: Dockerfile for Project 1.

Interview focus: Prepare to answer why Docker helps reproducibility in AI projects.

[Back to agenda](#)

Day 23: Run Project 1 In Docker

Goal: Get the main project working inside a container.

Study material

- [Docker Get Started](#)

Exact steps

- Build the image locally.
- Run the container and verify that the project still works.
- Document any environment variable or file path adjustments needed.

Suggested commands

```
cd projects/automation_support_agent
docker build -t automation-support-agent .
docker run --rm automation-support-agent
```

End-of-day output: Dockerized main project running locally.

Interview focus: Be ready to explain how containerization reduces machine-specific issues.

[Back to agenda](#)

Day 24: Dockerize Project 2

Goal: Apply the same packaging discipline to the second project.

Study material

- [Docker Get Started](#)

Exact steps

- Create a Dockerfile for the Log Analysis Agent.
- Build and run it locally.
- Compare what was reusable from Project 1.

Suggested commands

```
touch projects/log_analysis_agent/Dockerfile
cd projects/log_analysis_agent
docker build -t log-analysis-agent .
docker run --rm log-analysis-agent
```

End-of-day output: Both projects now containerized.

Interview focus: This shows repeatability, not just one-off demo work.

[Back to agenda](#)

Day 25: Add GitHub Actions CI

Goal: Introduce basic CI/CD awareness using GitHub Actions.

Study material

- [GitHub Actions Quickstart](#)

Exact steps

- Create ``.github/workflows/python-ci.yml``.
- Install dependencies and run a smoke test on push.

- Commit and verify that the workflow passes on GitHub.

Suggested commands

```
mkdir -p .github/workflows
touch .github/workflows/python-ci.yml
git add .github/workflows/python-ci.yml
git commit -m 'Add GitHub Actions CI'
git push
```

End-of-day output: Green CI pipeline on GitHub.

Interview focus: Prepare to describe CI as quality protection, not just automation for automation's sake.

[Back to agenda](#)

Day 26: Learn Hugging Face Spaces Deployment

Goal: Choose a simple deployment route for one demo.

Study material

- [Hugging Face Spaces Overview](#)

Exact steps

- Create a new Hugging Face Space.
- Decide whether you want a Gradio Space or a Docker Space.
- Prepare the chosen project for lightweight UI/demo deployment.

Suggested commands

```
touch .env.example .gitignore
code .env.example .gitignore
```

End-of-day output: Deployment target created and ready.

Interview focus: Know why demo deployment matters even if production deployment would differ.

[Back to agenda](#)

Day 27: Deploy One Project

Goal: Make one project publicly demonstrable.

Study material

- [Hugging Face Spaces Overview](#)

Exact steps

- Add a minimal UI if needed.
- Push the project to Hugging Face Spaces.
- Test the running app and save the live link.

Suggested commands

```
git add .
git commit -m 'Prepare app for HF Spaces deployment'
git push
```

End-of-day output: One live demo link.

Interview focus: This is useful proof that you can move beyond local-only prototypes.

[Back to agenda](#)

Day 28: Context Engineering Pass

Goal: Improve prompts and tool context using structured inputs.

Study material

- [Hugging Face Context Course](#)

Exact steps

- Read Unit 1 and Unit 2 overview with focus on context quality.
- Update prompts to include allowed actions, current evidence, and safe fallback behavior.
- Retest one scenario from each project.

Suggested commands

```
code projects/automation_support_agent/prompts.py
code projects/log_analysis_agent/prompts.py
```

End-of-day output: Context-optimized prompts and cleaner outputs.

Interview focus: Prepare to explain why context quality is often more important than model cleverness.

[Back to agenda](#)

Day 29: Interview And System Design Preparation

Goal: Convert the work into strong interview storytelling.

Study material

- [OpenAI Agents Guide](#)
- [Introducing AgentKit](#)

Exact steps

- Write answers for 20 interview questions.
- Prepare 3 system design diagrams: Automation Support Agent, Log Analysis Agent, Packaging Assistant idea.
- Practice explaining architecture, guardrails, deployment, and monitoring.

Suggested commands

```
touch notes/top-20-interview-answers.md
code notes/top-20-interview-answers.md
```

End-of-day output: `notes/top-20-interview-answers.md` and 3 diagrams.

Interview focus: This is where your profile starts to look intentionally positioned, not accidental.

[Back to agenda](#)

Day 30: Final Polish And Portfolio Pack

Goal: Finish with a clean, credible, interview-ready package.

Study material

- [Getting started with Git](#)
- [GitHub Actions Quickstart](#)

Exact steps

- Clean both repositories.
- Verify Docker, CI, and the live demo.
- Update resume bullets with project outcomes.
- Review all notes and rehearse your project walkthroughs.

Suggested commands

```
git status
docker images
python projects/automation_support_agent/app.py
python projects/log_analysis_agent/app.py
git add . && git commit -m 'Final roadmap deliverables complete'
git push
```

End-of-day output: Two repos, one live demo, one CI setup, and interview notes ready.

Interview focus: You should now be able to present yourself as an Automation Engineer transitioning into Agentic AI with practical proof.

[Back to agenda](#)